

**KOTLIN /  
Everywhere  
Nairobi**

# **WELCOME**

**Study Jam 4 / 6**



# Concurrency and Kotlin Coroutines

**JABEZ MAGOMERE**  
Student Developer



# Presentation Flow

Why Concurrency?

What is Concurrency Programming?

Concurrency vs Parallelism vs Sequential

Approaches to Concurrency

History of Coroutines

Introduction to Kotlin Coroutines

Hello Coroutine

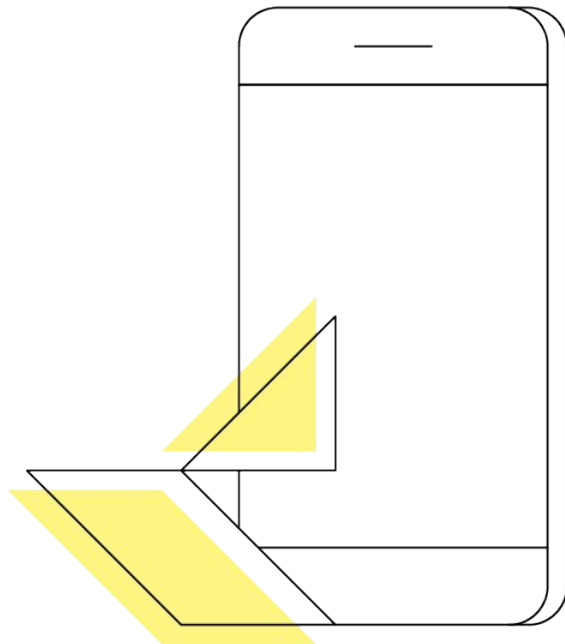
Suspending functions

Coroutine Builders

Coroutine Scopes

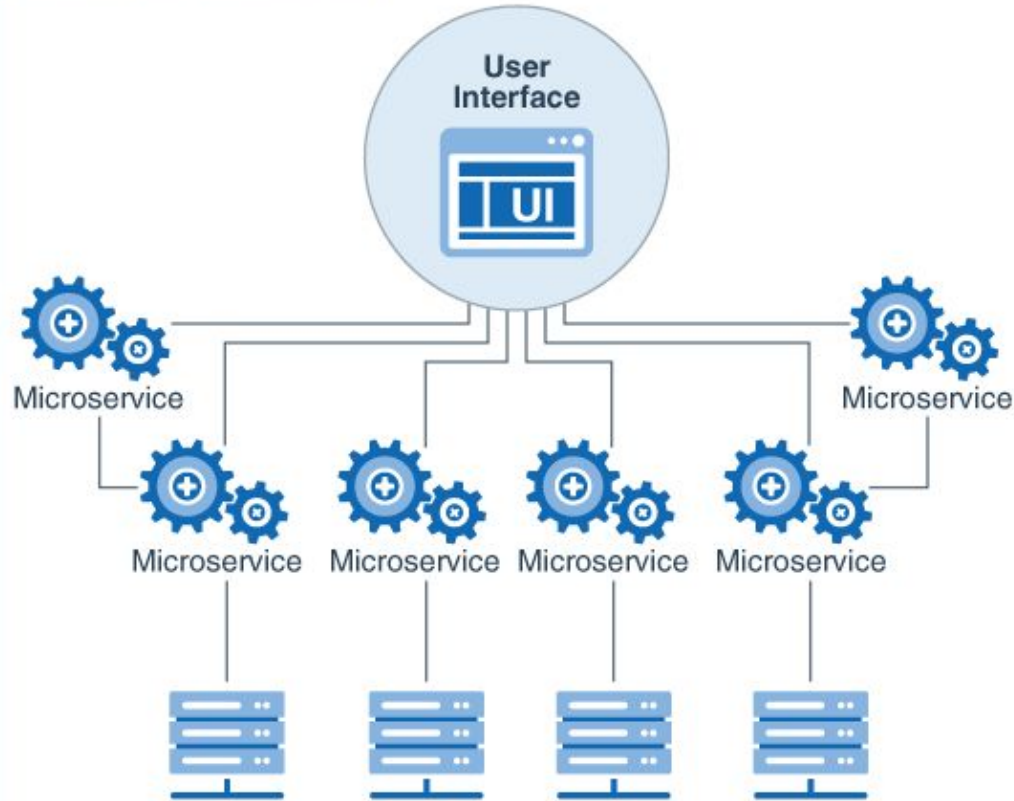
Dispatchers

Structured Concurrency

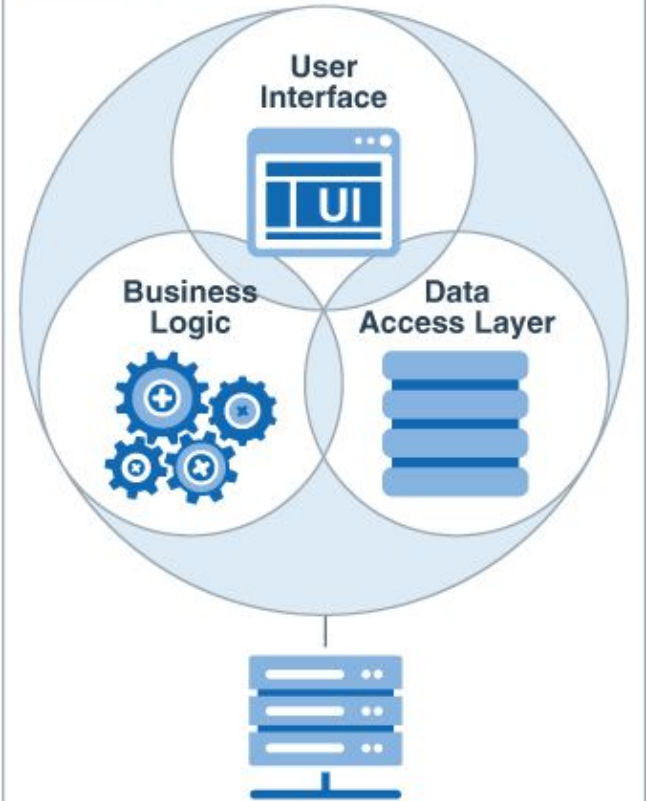


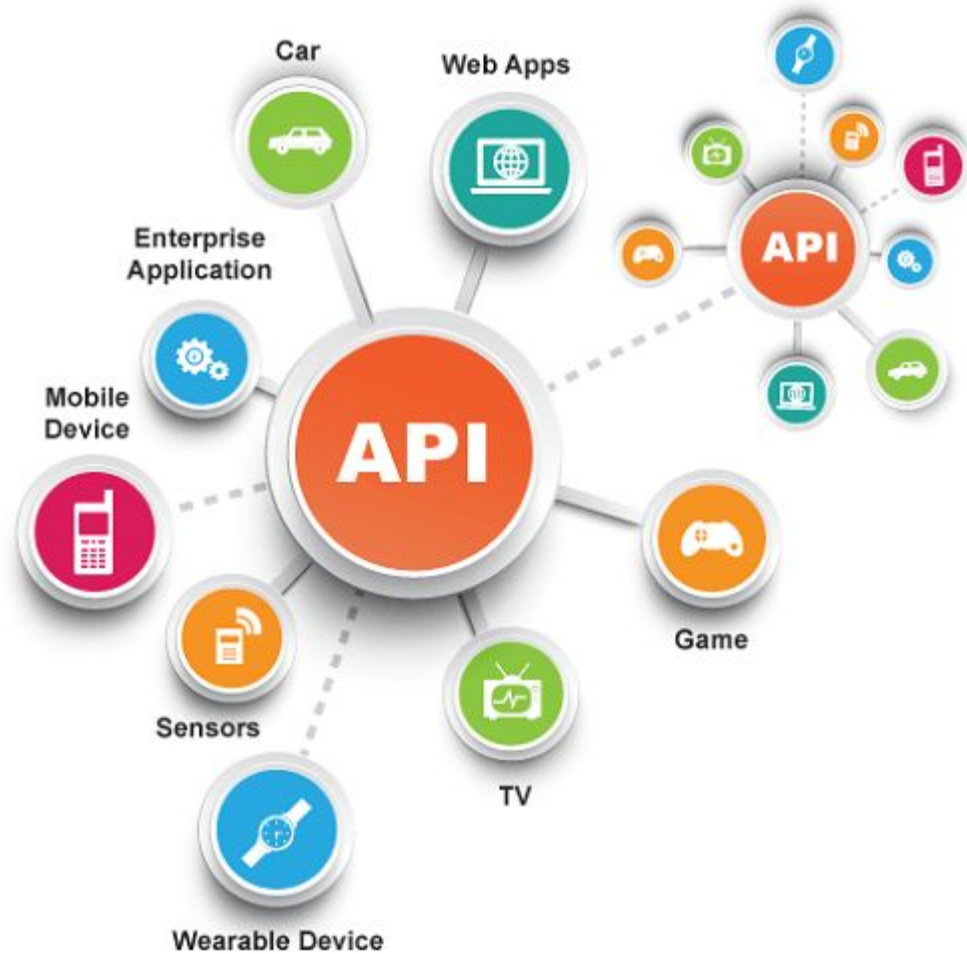
# Modern Systems

Microservice Architecture



Monolithic Architecture





## Why Concurrency

UI driven applications such as mobile applications or desktop applications have a UI thread that is commonly referred to as main thread.

The main thread is responsible for handling UI interactions and events e.g. rendering views and listening for user actions.

Performing cpu intensive or blocking operations on the main thread is catastrophic for User Experience.

Examples of resource intensive operations; fetching data from a remote data source(network requests), reading input from a file, performing complex calculations.

The need to perform multiple operations at the same time (time interval) results in the need for concurrency.



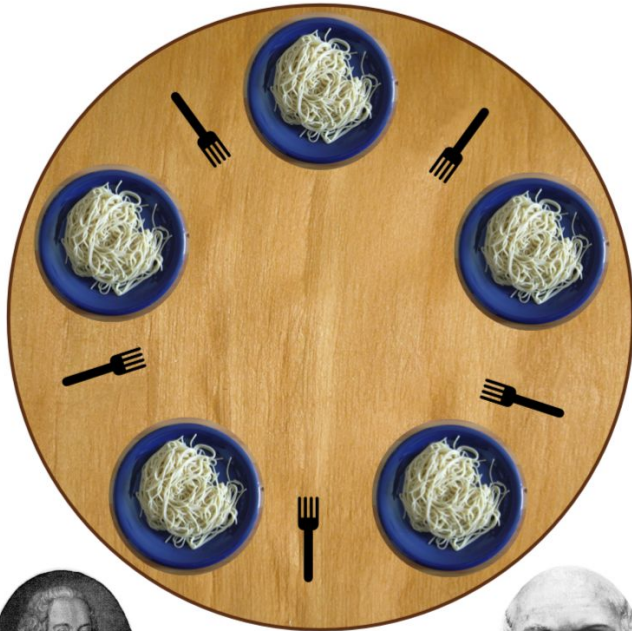
## What is Concurrency

Concurrency is applicable when discussed in the context of two or more tasks.

When an application is able to execute two or more tasks virtually at the same time. It is said to be a **concurrent** application.

The tasks are made to appear to run simultaneously or in parallel however concurrency leverages upon CPU time slicing, where a part of a task is run then it goes to waiting freeing the CPU to run another tasks, Achieved through interleaving of operations.





## What is Parallelism

Parallelism does not require two tasks to exist. In parallelism parts of a task or multiple tasks are run at the same time using multiple cores.

Assigns each core to one task or subtask.

Parallelism cannot exist in a single core.



## Concurrency vs Parallelism

Concurrency is when two tasks can start, run, and complete in overlapping time periods. Parallelism is when tasks literally run at the same time, eg. on a multi-core processor.

Concurrency is the composition of independently executing processes, while parallelism is the simultaneous execution of (possibly related) computations.

Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once.



An application can be concurrent – but not parallel, which means that it processes more than one task at the same time, but no two tasks are executing at same time instant.

An application can be parallel – but not concurrent, which means that it processes multiple sub-tasks of a task in multi-core CPU at same time.

An application can be neither parallel – nor concurrent, which means that it processes all tasks one at a time, sequentially.

An application can be both parallel – and concurrent, which means that it processes multiple tasks concurrently in multi-core CPU at same time



# Achieving Concurrency

## Threading



```
fun postItem(item: Item) {  
    val token = preparePost()  
    val post = submitPost(token, item)  
    processPost(post)  
}  
  
fun preparePost(): Token {  
    // makes a request and consequently blocks the main thread  
    return token  
}
```



## Drawbacks of Threading

Creating threads is expensive because of context switching.

Threads are not infinite. The number of threads that can be created is finite depending on the underlying operating system.

Threads aren't always available e.g. JavaScript.

Debugging threads is not easy. '*Kwa ground vitu ni different*' i.e. deadlocks, race conditions.



## Callbacks

The idea of a callback is to pass a function as a parameter to another function so that it is invoked after the operation completes.

```
fun postItem(item: Item) {  
    preparePostAsync { token ->  
        submitPostAsync(token, item) { post ->  
            processPost(post)  
        }  
    }  
}  
  
fun preparePostAsync(callback: (Token) -> Unit) {  
    // make request and return immediately  
    // arrange callback to be invoked later  
}
```



## Drawbacks of Callbacks

**Callback hell** as a result of nesting callbacks.

Error handling is difficult.



## Future, Promises et.al

The idea is that when a call is made the function will at some point return an object called promise, which can then be operated on.

```
fun postItem(item: Item) {  
    preparePostAsync()  
        .thenCompose { token ->  
            submitPostAsync(token, item)  
        }  
        .thenAccept { post ->  
            processPost(post)  
        }  
}  
  
fun preparePostAsync(): Promise<Token> {  
    // makes request and returns a promise that is completed later  
    return promise  
}
```



## Reactive Extensions (Rx)

The idea behind Rx is to move towards observable streams, data is seen as streams (infinite amount of data) and these streams can be observed.

**"everything is a stream, and it's observable"**



# Kotlin

## Coroutines



***In the beninging....***



# Introduction to Coroutines

The concept of coroutines has existed historically in other programming languages such as python, C#, Ruby, Go and JavaScript.

Coroutines is a powerful concept that allows us to achieve asynchronous programming. The world is not sequential, multiple events happen at the same time, e.g. systems communicating, hence the need for concurrency.

Coroutines are light weight threads launched with a **coroutine builder** in a context of some sort referred to as a **coroutine cscope**. The lifeline of the coroutine is limited to the scope in which it is launched.



# Don't block Keep moving

---

A Tasks — Threads





# Structured Concurrency

Keeping track of all coroutines manually can be a tiresome task. Poor code can result to a work leak.

A leaked coroutine can waste memory , CPU disk or even launch a request that's not needed. Structured concurrency ideally helps to prevent coroutines from leaking.

Structured concurrency is a combination of language features and best practices that when followed, help you keep track of all work running in coroutines.

## Goals of Structured Concurrency

- Cancel work that is no longer needed.
- Keep track of work while it's running.
- Signal errors when a coroutine fails.



# Guarantees of Structured Concurrency

1. When a scope cancels, all of its coroutines cancel.
2. When a suspend fun returns, all of its work is done.
3. When a coroutine errors, its caller or scope is notified.



# Coroutine Scope

- Defines a new scope for coroutines.
- Scopes help us to launch coroutines on entities with a well-defined lifecycle. E.g. Application Lifecycle
- A coroutine scope keeps track of all the coroutines started in it and it can cancel the coroutines started in it. In android, it makes sense to associate a coroutine scope with a user view. Kotlin does not allow you to start a new coroutine without a coroutine scope.



# Global Scope

It's a general scope that can be used for any coroutines that are meant to continue executing while the App is running. Coroutines launched in this scope are not tied to any components that can be destroyed. However, it is advisable not to use the global scope in android applications.

```
GlobalScope.launch(Dispatchers.Main) {  
  
    /**  
    perform long running operations, e.g fetch data from remote source  
    **/  
  
}
```



# Suspending Functions

- Coroutines are based on suspending functions.
- Suspend keyword is used to mark functions that perform blocking operations. e.g network requests, writing to a file or performing a resource intense computation.
- Suspend functions may suspend the execution of the current coroutine without blocking the main thread. Suspend keyword allows us to write asynchronous code sequentially.
- Suspend functions can only run inside another suspend function or a coroutine.
- Suspend functions do not have any special return types such as; Future or Promise.

```
/**
```

```
Marking this function as suspend implies that it may be suspended and  
later on resumed because it performs a long running task
```

```
*/
```

```
suspend function fetchAllUsers():List<User>{  
    return userRepository.fetchAllusers()  
}
```



# Suspend and Resume

Coroutines build upon regular functions by adding two new operations. In addition to `invoke`(or `call`) and `return`, coroutines add **suspend** and **resume**. Suspend and Resume work together to replace callbacks.

**Suspend**- pause the execution of the current coroutine, saving all local variables.

**Resume**- continue a suspended function from the place it was paused.

When all coroutines are suspended the main thread is free to do other work.



```
// Dispatchers.Main
suspend fun fetchDocs() {
    // Dispatchers.IO
    val result = get("developer.android.com")
    // Dispatchers.Main
    show(result)
}

suspend fun get(url: String) = withContext(Dispatchers.IO){/*...*/}
```



```
suspend fun fetchDocs() {  
    val docs = get("...")  
    show(docs)  
}
```

suspend

resume

Main Thread  
[stack]



## withContext

This is a function that allows to easily change the context that will be used to run a part of the code inside a coroutine. The section of code wrapped inside withContext block will be run on the specified dispatcher. Allows you to specify what thread executes what block of code. It ensures that every function is safe to be called on any Dispatcher, including main.

```
suspend fun get(url: String) =  
    // Dispatchers.IO  
    withContext(Dispatchers.IO) {  
        // Dispatchers.IO  
        /* perform blocking network IO here */  
    }  
    // Dispatchers.Main
```



# Coroutine Builders

Coroutine builders are functions that create a new coroutine to run a suspending function.

Examples of coroutine builders: `runBlocking`, `launch`, `async`.



# runBlocking

Blocks the current thread until all the tasks inside that coroutine are finished.

```
fun main() {  
    println("Hello,")  
    //block the main thread while waiting for the coroutine to finish its execution  
    runBlocking {  
        delay(2000L) // suspends the current coroutine for 2 seconds  
    }  
    // will be executed after 2 seconds  
    println("World!")  
}
```



# launch

Launches a new coroutine without blocking the current thread and returns a reference to the coroutine as a job. The coroutine is cancelled when the job is cancelled. Also referred to as “fire and forget”- it won't return the result to the caller.

```
//Under the hood
fun CoroutineScope.launch(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> Unit
): Job { source }
```



## launch continued ...

The builder always needs a scope ( To be discussed ahead).



```
GlobalScope.launch(Dispatchers.Main) {  
    ...  
}
```



## async

Creates a coroutine and runs its future result as an implementation of deferred similar to *Future* or *Promise*. The main difference between `async` and `launch` is that `async` returns a deferred object which you can `await` to get the result of the operation unlike `launch` which returns a job object. `Async` does not throw exceptions by default since it will return a result or exception to the caller.

```
GlobalScope.launch(Dispatchers.Main) {  
  
    val currentFriends = async(Dispatchers.IO) { userService.requestCurrentFriends(user) }  
    val suggestedFriends = async(Dispatchers.IO) { userService.requestSuggestedFriends(user) }  
  
    val allFriends = currentFriends.await()+suggestedFriends.await()  
  
}
```



**[http://cf0385d2.ngrok.io/  
movies](http://cf0385d2.ngrok.io/movies)**



# Dispatchers

A coroutine dispatcher determines what thread a corresponding coroutine uses for its execution. There are three main types of dispatchers: `Dispatchers.Main`, `Dispatchers.IO`, `Dispatchers.Default`.

| <code>Dispatchers.<u>Main</u></code>                                                                                                | <code>Dispatchers.<u>IO</u></code>                                                                                 | <code>Dispatchers.<u>Default</u></code>                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Main thread on Android, interact with the UI and perform light work                                                                 | Optimized for disk and network IO off the main thread                                                              | Optimized for CPU intensive work off the main thread                                                        |
| <ul style="list-style-type: none"><li>- Calling suspend functions</li><li>- Call UI functions</li><li>- Updating LiveData</li></ul> | <ul style="list-style-type: none"><li>- Database*</li><li>- Reading/writing files</li><li>- Networking**</li></ul> | <ul style="list-style-type: none"><li>- Sorting a list</li><li>- Parsing JSON</li><li>- DiffUtils</li></ul> |